

A Functionally Self-Testable RISC-V Processor with a Custom Test Instruction Set

Abstract—Ensuring reliable processor operation is critical in modern computing systems, especially in scenarios where faults may arise during runtime. Traditional approaches such as Built-In Self-Test (BIST) and Software-Based Self-Test (SBST) either incur significant hardware overhead or lack flexibility for dynamic testing.

This work presents a self-test capable RISC-V processor that enables on-demand functional validation using a lightweight architecture. A custom instruction is introduced to trigger test mode, allowing ATPG-generated test patterns stored in memory to be executed through the native datapath. A dedicated control FSM coordinates test execution, while a comparator and pass/fail counters provide efficient result evaluation.

The design ensures non-intrusive operation through a program counter save and restore mechanism, preserving the processor state during testing. Experimental validation on a single-cycle RISC-V processor demonstrates effective fault detection with minimal hardware overhead. The proposed approach is scalable and suitable for in-field testing of processor-based systems.

Index Terms—RISC-V, self-test processor, ATPG, fault detection, SBST, processor testing, hardware validation

I. INTRODUCTION

Modern processor-based systems are widely deployed to execute long-running and computation-intensive workloads across a variety of domains, including scientific computing, data analytics, and embedded applications. In such environments, correctness of computation is of paramount importance, as even a minor hardware fault can propagate through execution and lead to incorrect results, system instability, or significant loss of computational resources. This risk becomes more critical when processors are used for large-scale or mission-critical tasks, where failures may not be immediately observable but can have long-term consequences.

Despite the importance of hardware reliability, most practical workflows do not provide users with a mechanism to verify processor correctness prior to execution. Processors are typically assumed to be fault-free once deployed, and validation is largely limited to manufacturing-time testing and initial system verification. However, hardware faults may arise during operation due to factors such as aging, environmental conditions, or transient disturbances. As a result, relying solely on pre-deployment testing introduces a gap between assumed and actual system reliability, making systems vulnerable to silent failures.

Existing approaches for processor validation can be broadly categorized into hardware-based and software-based techniques. Built-In Self-Test (BIST) is a widely adopted hardware-based method that incorporates dedicated test circuitry within the processor to enable self-testing [1]. While

BIST provides structured and reliable fault detection, it introduces additional hardware overhead in terms of area, power, and design complexity. Moreover, such hardware support is fixed at design time and does not offer flexibility for user-level invocation or customization of testing procedures during normal operation.

Software-Based Self-Test (SBST), on the other hand, offers a more flexible alternative by utilizing instruction-level programs to test hardware functionality [2], [3]. In SBST, specially designed test routines are executed on the processor to activate potential faults and propagate their effects to observable outputs. This approach eliminates the need for extensive hardware additions and enables at-speed testing using existing processor resources. However, conventional SBST techniques often rely on handcrafted test programs or require detailed knowledge of the processor’s internal structure, making them difficult to scale across different implementations. Furthermore, generating high-quality test programs that achieve significant fault coverage remains a challenging task.

Recent works have explored testing methodologies for RISC-V processors using software-based test libraries [4]. While these approaches improve portability and automation, they still lack a unified mechanism that allows users to invoke structured testing dynamically during runtime without external dependencies.

Another limitation of existing methodologies is their lack of accessibility to end users. In most systems, testing is either confined to manufacturing stages or requires specialized tools and expertise, making it impractical for users to perform validation before executing critical workloads. Consequently, users are forced to rely on the assumption of correct hardware behavior, which may not always hold true in real-world scenarios.

These challenges highlight the need for a lightweight, flexible, and user-invokable testing mechanism that can operate within the processor itself and provide reliable validation without incurring significant overhead. Such a mechanism should enable users to verify processor correctness dynamically, prior to executing important tasks, while maintaining compatibility with existing processor architectures.

To address this need, this work proposes the concept of a *self-test capable processor*, which enables on-demand hardware validation through a memory-driven execution framework. The proposed approach integrates a structured test mechanism within the processor, where predefined test patterns are stored in a reserved memory region and executed using the native datapath. By leveraging existing computational

units, the framework ensures that testing is performed at speed and under realistic operating conditions.

The proposed architecture is designed with minimal hardware overhead, requiring only a small set of additional components, including registers for operand and expected value storage, a comparator for result verification, and counters for tracking test outcomes. This lightweight design ensures that the testing mechanism does not significantly impact the performance or area of the processor, while still providing meaningful fault detection capability.

Overall, the proposed approach provides a practical solution for improving processor reliability by enabling dynamic, user-level validation with low overhead. By combining structured test pattern execution with minimal architectural modifications, the framework bridges the gap between traditional testing methodologies and real-world usability, making it suitable for deployment in modern processor-based systems.

II. PROPOSED IDEA AND ARCHITECTURE

The proposed work introduces a self-test capable processor architecture that enables on-demand validation of processor functionality prior to executing critical workloads. The architecture integrates a lightweight testing capability within the processor, allowing users to invoke hardware validation dynamically without requiring external testing infrastructure or complex hardware additions.

A. System-Level Overview

The proposed framework follows a memory-driven self-test approach, where predefined test patterns are stored in a reserved memory region and executed using the processor datapath under the supervision of a dedicated control unit. These test patterns are generated using Automatic Test Pattern Generation (ATPG) techniques [5], [6], ensuring high fault coverage at the structural level while being executed through the normal functional hardware path.

The complete framework consists of the following components:

- Reserved test memory for storing ATPG-derived test patterns
- A custom test instruction for invoking test mode
- A dedicated test control FSM
- Test registers for operand and expected-value storage
- A comparator for result verification
- Pass and fail counters for reporting test outcome
- A PC save/restore register for preserving execution state

B. Custom Test Instruction

To enable seamless switching between normal execution and test execution, a custom instruction is introduced using opcode 0001011. In addition to the 7-bit opcode, 4 bits are reserved to identify the type of test to be executed, resulting in 11 bits used for test invocation.

In the current implementation, the 4-bit test-type field selects the unit under test. The encoding 0000 is used for ALU testing, while other encodings can be used for additional functional units in future extensions.

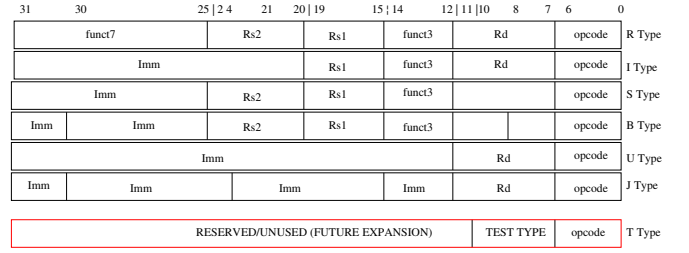


Fig. 1. RISC-V ISA format and our proposed custom test instruction format shown as T Type. The opcode 0001011 triggers the transition to test mode, while the 4-bit test-type field selects the target unit under test. The remaining bits are reserved for future extensions such as configuration and control parameters.

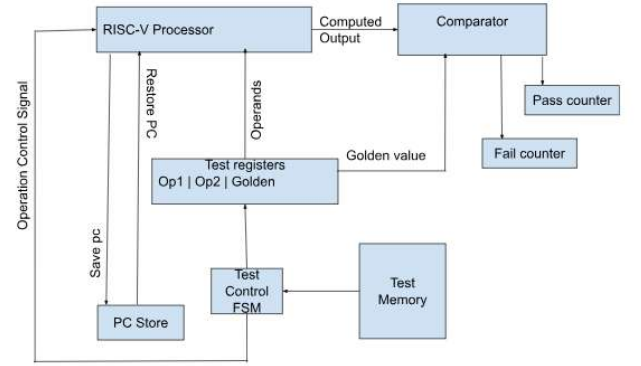


Fig. 2. Integration of the proposed self-test framework into a RISC-V processor. Test patterns are fetched from memory, applied through dedicated test registers, and verified using a comparator and counters. A PC save/restore mechanism ensures non-intrusive execution.

C. Test Pattern Representation and Storage

Test patterns are derived from ATPG-generated vectors and formatted for execution within the processor. Each test entry consists of four elements:

- Operand 1
- Operand 2
- Operation Control Signal
- Expected (Golden) Result

These test entries are stored sequentially in a reserved test memory region.

To indicate the completion of the test sequence, a predefined end marker is used. In the current implementation, the value 32'hFFFFFFF is used as the end marker. Upon encountering this value during test execution, the processor exits test mode.

D. Architecture Integration

As shown in Fig. 2, the proposed self-test framework is integrated into a standard RISC-V processor by augmenting it with test memory, a test control FSM, test registers, a comparator, pass/fail counters, and a PC save/restore register.

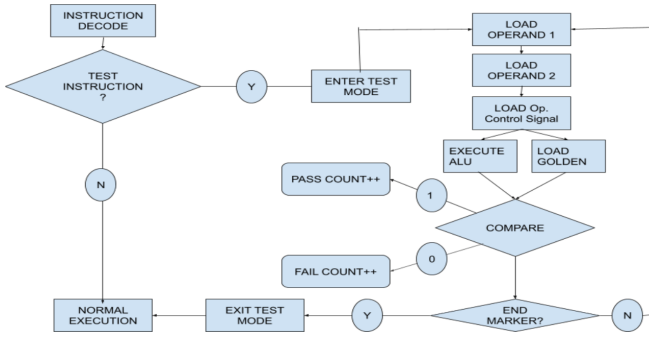


Fig. 3. Self-test execution flow of the proposed processor architecture. The processor enters test mode upon detecting the custom instruction, executes test patterns sequentially, compares results, and exits upon detecting the end marker.

When the processor detects the custom test instruction, it transitions into test mode and saves the current program counter (PC+4) to preserve execution state.

The test control FSM acts as the central coordinator of the self-test process. It fetches test patterns from memory, loads operands and expected values into test registers, and generates the required control signals to drive the processor datapath.

The test registers supply operand values to the processor, while the expected value is forwarded to the comparator. The computed output from the processor is compared with the expected result, and the outcome is recorded using pass and fail counters. The FSM monitors the test sequence and checks for the end marker to terminate execution.

E. Non-Intrusive Test Execution

A key design objective of the proposed architecture is to ensure that test execution does not affect the normal execution state of the processor.

Before entering test mode, the current PC value is stored in a dedicated register. After completion of testing, this value is restored so that the processor resumes execution from the same point.

Additionally, test execution is isolated from the architectural state. Test computations are performed using dedicated test registers, ensuring that the register file and memory contents used during normal execution remain unchanged.

F. Test Execution Flow

The complete test execution flow is illustrated in Fig. 3. The processor operates in normal mode until the custom test instruction is detected, after which it transitions into test mode.

Each test pattern is executed through the following sequence:

- 1) Load Operand 1
- 2) Load Operand 2
- 3) Load operation control signal
- 4) Execute operation using processor datapath
- 5) Load expected result
- 6) Compare computed and expected outputs

After each test, the system checks for the end marker. If not encountered, the next test pattern is executed. Otherwise, the processor exits test mode, restores the saved PC, and resumes normal execution.

G. Result Verification Mechanism

A comparator is used to verify correctness by comparing computed outputs with expected values. The results are recorded using:

- Pass Counter
- Fail Counter

H. Hardware Overhead

The proposed architecture introduces minimal hardware overhead. It requires only a small test memory, a test control FSM, test registers, a comparator, pass/fail counters, and a PC save/restore register.

No complex scan chains or large BIST structures are required, making the approach lightweight, scalable, and suitable for real-time validation.

III. EXPERIMENTAL RESULTS

The proposed self-test framework has been implemented on a single-cycle RISC-V processor to validate its functionality and effectiveness. The implementation focuses on ALU-level testing using ATPG-derived test patterns executed through the processor datapath.

A. Test Memory Organization

The test patterns are stored in a reserved region of the instruction memory. As shown in Fig. 4, normal program instructions occupy the lower memory addresses, while the test memory region begins at address 0x800.

Each test case is organized into four consecutive 32-bit memory locations corresponding to:

- Operand 1
- Operand 2
- Operation Code
- Expected (Golden) Result

This structured format allows direct mapping of ATPG-generated test vectors onto the processor execution flow without requiring additional decoding logic.

In the implemented prototype, a total of 888 test cases were used for ALU validation. Since each test case occupies 16 bytes, the test memory region extends up to address 0x3F80. The final entry in memory is reserved for the end marker, defined as 0xFFFFFFFF, which signals termination of the test sequence.

B. ATPG-Based Fault Coverage Analysis

To evaluate the effectiveness of the proposed self-test framework, Automatic Test Pattern Generation (ATPG) was applied to the ALU module to generate deterministic test patterns targeting structural faults. These patterns were then executed using the proposed test mechanism through the processor datapath.

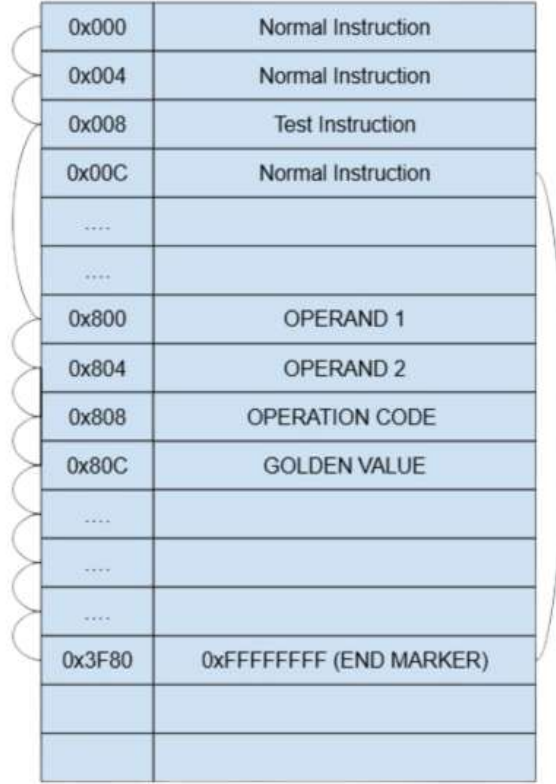


Fig. 4. Memory organization of test patterns in the implemented processor. A reserved region starting at address 0x800 stores ATPG-generated test patterns, with each test occupying four consecutive entries. The sequence terminates with an end marker at address 0x3F80 for the prototype implementation.

TABLE I
ATPG-BASED FAULT COVERAGE RESULTS FOR ALU

Parameter	Value
Target Module	ALU
Total Faults	(6580)
Detected Faults	(6489)
Fault Coverage (%)	(98.61)
Test Patterns Generated	888
Pattern Type	Deterministic (ATPG)
Fault Model	Stuck-at Fault Model

Table I summarizes the fault coverage achieved using ATPG for the ALU module. We have used DesignCompiler and TestMax for synthesis and ATPG process on our ALU.

C. Test Execution and Validation

The self-test mechanism is triggered using the custom test instruction. Upon detection, the processor transitions into test mode, and the test control FSM begins sequential execution of test patterns from the reserved memory region.

For each test case, operand values are loaded into dedicated test registers and applied to the processor datapath. The computed result is compared with the expected value using a

comparator unit. Based on the comparison outcome, the pass or fail counter is incremented.

The FSM continuously monitors the memory contents for the end marker. Upon encountering the value 0xFFFFFFFF, the test sequence is terminated, and the processor exits test mode. The previously stored program counter is restored, ensuring seamless continuation of normal execution.

D. Hardware Overhead and Observations

The implementation demonstrates that the proposed self-test framework can be integrated with minimal hardware overhead. The additional components required include a small test memory region, a test control FSM, test registers, a comparator, pass/fail counters, and a PC save/restore register.

Despite its simplicity, the framework effectively validates processor functionality by executing structurally generated test patterns at the functional level. The use of memory-based test storage and instruction-triggered execution makes the approach scalable and suitable for in-field testing scenarios.

The experimental results confirm that the proposed architecture successfully detects faults in ALU operations while maintaining non-intrusive execution, as the normal processor state remains unaffected during the test process.

IV. CONCLUSION AND FUTURE WORK

The current implementation demonstrates the feasibility of integrating a self-test framework within a single-cycle RISC-V processor for ALU-level validation. While the results are promising, several extensions can be explored to enhance the capability and applicability of the proposed approach.

One immediate direction is to extend the framework to support multi-cycle and pipelined processor architectures. In such designs, additional considerations such as pipeline hazards, stage synchronization, and control signal timing must be addressed to ensure correct test execution without disrupting normal operation.

Another important extension involves expanding the test coverage beyond the ALU to include other critical components such as the control unit, register file, and memory subsystem. The existing test-type field in the custom instruction can be utilized to selectively target different units, enabling a more comprehensive self-test mechanism.

The current design utilizes a portion of the instruction encoding (7-bit opcode and 4-bit test-type field), leaving the remaining bits available for future enhancements. These unused bits can be leveraged to encode advanced features such as test prioritization, selective execution, configurable test modes, or fault classification mechanisms.

In the present prototype, approximately 14 KB of memory is used to store test patterns. Although this may appear significant for a small-scale processor, the memory requirement scales linearly with the number of test cases and remains manageable for larger systems. Future work can explore compression techniques and optimized test pattern storage to further reduce memory overhead.

Finally, integration with automated ATPG tools and fault injection frameworks can enable more systematic evaluation of fault coverage and reliability. This would allow the proposed architecture to evolve into a robust in-field testing solution suitable for safety-critical and high-performance computing systems.

REFERENCES

- [1] P. H. Bardell, W. H. McAnney, and J. Savir, *Built-In Test for VLSI: Pseudorandom Techniques*. Wiley, 1987.
- [2] T. Faller, N. I. Deligiannis, R. Cantoro, M. S. Reorda, and B. Becker, “Special session: Software-based self-test generation for risc-v – stuck-at generation, functional cell-aware untestability, and fpga demonstration –,” in *2024 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2024, pp. 1–6.
- [3] M. Psarakis, D. Gizopoulos, E. Sanchez, and M. S. Reorda, “Microprocessor software-based self-testing,” *IEEE Design & Test of Computers*, 2010.
- [4] L. C. et al., “Testing risc-v-based cores with software test libraries,” in *IEEE*, 2020.
- [5] M. Abramovici, M. A. Breuer, and A. D. Friedman, *Digital Systems Testing and Testable Design*. IEEE Press, 1990.
- [6] N. K. Jha and S. Gupta, *Testing of Digital Systems*. Cambridge University Press, 2003.